UNIVERSIDADE CESUMAR – UNICESUMAR

Curso de Engenharia de Software – ESOFT2S

MATHEUS GUELLES DONÁ
JOÃO ARTUR PRADO MARTINS
NICHOLAS TADEI ESTEVES

PSE EM AÇÃO

Sistema de apoio à vacinação escolar no Programa Saúde na Escola

Link do repositório do Github
https://github.com/MatheusDona/AEP_Segundo_Semestre.git

2026

MATHEUS GUELLES DONÁ
JOÃO ARTUR PRADO MARTINS
NICHOLAS TADEI ESTEVES

PSE EM AÇÃO

Sistema de apoio à vacinação escolar no Programa Saúde na Escola

Atividade Extensionista Prática (AEP) – Etapa 2: implementação e apresentação, apresentada ao curso de Engenharia de Software da Universidade Cesumar – UniCesumar, referente ao 2º semestre de 2026.

2026

1 RETOMADA DO PROJETO E OBJETIVOS

Este documento reúne os resultados da **Etapa 2** da Atividade Extensionista Prática (AEP): a implementação, em linguagem C, do sistema **PSE em Ação**, planejado na Etapa 1, junto com os artefatos de engenharia de software revisados, o registro de testes, o manual do usuário e as considerações finais. Os requisitos (RF e RNF) da Etapa 1 foram mantidos, de modo que cada decisão possa ser rastreada do planejamento até o código.

1.1 Problema de pesquisa e contexto

O Programa Saúde na Escola (PSE) foi instituído pelo Decreto nº 6.286, de 5 de dezembro de 2007, para contribuir com a formação integral dos estudantes da rede pública por meio de ações de promoção, prevenção e atenção à saúde (BRASIL, 2007). Entre suas ações prioritárias está a verificação da situação vacinal dos estudantes, com encaminhamento dos casos incompletos às salas de vacina (BVS/MS, [s.d.]). Apesar do avanço na integração entre Saúde e Educação, estudos apontam fragilidades na participação equilibrada dos setores e a permanência de ações pontuais e pouco sistematizadas (SOUSA; ESPERIDIÃO; MEDINA, 2017). A contextualização completa está na Etapa 1.

A questão norteadora que orientou o desenvolvimento foi mantida:

Como uma aplicação desenvolvida em linguagem C pode auxiliar uma equipe escolar e de saúde a planejar, registrar e acompanhar ações de vacinação no âmbito do Programa Saúde na Escola, apresentando informações organizadas e preservando a privacidade dos estudantes?

1.2 Objetivos e atendimento na implementação

Objetivo geral: desenvolver um sistema em linguagem C, executado em terminal, para apoiar o planejamento e o acompanhamento de ações de vacinação escolar realizadas no contexto do Programa Saúde na Escola.

Quadro 1 – Objetivos específicos e como foram atendidos

Objetivo específico (Etapa 1)	Como foi atendido na Etapa 2
Estruturar o cadastro de ações de vacinação com os dados mínimos necessários, sem coleta de dados individuais sensíveis.	Função `cadastrarAcao` e `struct Acao`, com código, escola, tema, data, público-alvo, responsável e quantidade prevista. Nenhum campo identifica estudantes (RF01, RNF03).
Implementar consulta e atualização por código, escola ou tema e registrar a situação (planejada, realizada ou cancelada).	Funções `pesquisarAcao` e `atualizarSituacao`, com busca por código exato ou por trecho de escola/tema e registro dos participantes efetivos (RF03, RF04).

Objetivo específico (Etapa 1)	Como foi atendido na Etapa 2
Gerar relatórios consolidados com a quantidade de ações por situação e o percentual de participação em relação ao previsto.	Função `gerarResumo`, que apresenta ações por situação, total de participantes, percentual de participação e total por escola (RF05).

2 REQUISITOS REVISADOS

2.1 Usuários e escopo

Usuários: profissionais da equipe de saúde (Atenção Primária) e educadores responsáveis pela execução do PSE na escola, que usam o sistema para planejar e acompanhar ações coletivas de vacinação. Não há perfis diferentes de acesso: o programa é de uso local, em terminal, por um operador por vez.

Dentro do escopo: cadastrar, listar, pesquisar, atualizar a situação e resumir ações coletivas de vacinação, com dados fictícios mantidos em memória durante a execução.

Fora do escopo: dados individuais de estudantes (nome, diagnóstico, prontuário, condição clínica, carteira de vacinação individual), diagnóstico, triagem, prescrição, banco de dados, interface gráfica, aplicação web e persistência em arquivo.

2.2 Requisitos funcionais

Quadro 2 – Requisitos funcionais (RF)

Código	Descrição
RF01	O sistema deve permitir o cadastro de uma ação de vacinação, contendo código, escola, data prevista, público-alvo, responsável, quantidade prevista de participantes e situação inicial (“planejada”).
RF02	O sistema deve listar todas as ações cadastradas, exibindo suas informações de forma organizada e legível no terminal.
RF03	O sistema deve permitir a pesquisa de ações por código, escola ou tema, retornando os registros correspondentes.
RF04	O sistema deve permitir a atualização da situação de uma ação (planejada, realizada ou cancelada) e, quando marcada como realizada, registrar a quantidade efetiva de participantes vacinados.
RF05	O sistema deve gerar um resumo geral contendo a quantidade de ações planejadas, realizadas e canceladas, o total de participantes vacinados e o percentual de participação em relação ao número previsto.
RF06	O sistema deve validar as entradas do usuário, impedindo códigos duplicados, quantidades negativas, campos obrigatórios vazios e opções inexistentes no menu.

2.3 Requisitos não funcionais

Quadro 3 – Requisitos não funcionais (RNF)

Código	Descrição
RNF01	O sistema deve ser executado em ambiente de terminal, sem uso de interface gráfica, banco de dados ou bibliotecas externas avançadas.
RNF02	O sistema deve armazenar os dados em memória (vetores/structs) durante a execução, sem persistência obrigatória em arquivo.

Código	Descrição
RNF03	O sistema não deve armazenar nome, diagnóstico, prontuário ou qualquer dado clínico individual de estudantes — apenas informações fictícias e coletivas sobre as ações.
RNF04	O código deve ser modularizado em funções com responsabilidades claras, evitando concentrar a lógica na função main.
RNF05	O sistema deve apresentar mensagens claras de confirmação e de erro, garantindo usabilidade mesmo para operadores sem conhecimento técnico.

2.4 Alterações em relação à Etapa 1

Os requisitos funcionais e não funcionais permanecem os mesmos da Etapa 1. Durante a implementação, porém, surgiram refinamentos de detalhe que melhoram a usabilidade e a coerência do sistema sem alterar a numeração nem o objetivo dos requisitos. Todos foram registrados abaixo com a respectiva justificativa.

Quadro 4 – Mudanças justificadas entre o planejamento e a implementação

Item	Etapa 1 (planejado)	Etapa 2 (implementado e motivo)
Tratamento de erro no cadastro	Qualquer erro de validação voltava ao início do cadastro.	Repete apenas o campo inválido. Evita que o operador redigite todos os dados (RNF05).
Validação de data	Não prevista.	Formato DD/MM/AAAA, com conferência de dias do mês e ano bissexto. Evita datas impossíveis na listagem.
Limite de armazenamento	Não prevista.	Máximo de 50 ações, com mensagem de erro. Exigido pela Etapa 2 e necessário ao usar vetor.
Situação da ação	Texto (“planejada”, “realizada”, “cancelada”).	Tipo `enum` (1, 2, 3), convertido em texto na exibição. Evita erros de digitação e simplifica as comparações.
Comparação de código	Igualdade simples.	Ignora maiúsculas/minúsculas (VAC-001 e vac-001 são o mesmo código), prevenindo duplicidade aparente.
Termo “participantes”	RF04 e RF05 falavam em “participantes vacinados”.	Passou a “participantes”. O sistema registra apenas a adesão coletiva, não quem foi vacinado, o que preserva a privacidade.
Percentual de participação	Em relação ao número previsto (sem detalhar a base).	Calculado sobre o previsto das ações realizadas. Ações planejadas ou canceladas não distorcem o índice.
Resumo por escola	Não prevista.	Total de participantes por escola, que atende à necessidade descrita no desafio de consolidar o atendimento de cada escola.
Participantes acima do previsto	Não prevista.	Aceito, com aviso ao usuário, pois a adesão real pode superar a previsão.

2.5 Rastreabilidade entre requisitos, código, casos de uso e testes

Quadro 5 – Rastreabilidade dos requisitos

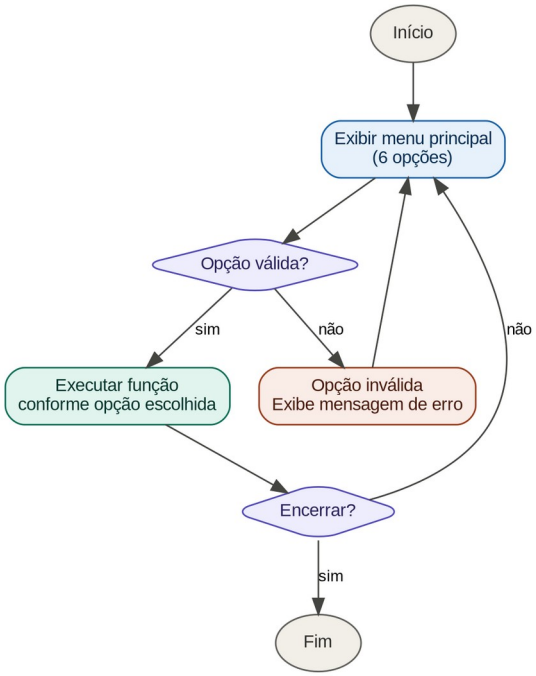
Req.	Funções em C (`src/main.c`)	Caso de uso	Casos de teste (Seção 6)
RF01	`cadastrarAcao`, `lerCodigoNovo`, `lerDataPrevista`, `lerQuantidade`	UC01	CT05 a CT12, CT29
RF02	`listarAcoes`, `exibirAcao`, `nomeSituacao`	UC02	CT13, CT14
RF03	`pesquisarAcao`, `igualIgnorandoCaixa`, `contemTexto`	UC03	CT15 a CT19
RF04	`atualizarSituacao`, `buscarPorCodigo`, `lerQuantidade`	UC04	CT20 a CT25
RF05	`gerarResumo`	UC05	CT26 a CT28
RF06	`lerLinha`, `lerTextoObrigatorio`, `lerInteiro`, `dataValida`, `buscarPorCodigo`	UC06	CT01 a CT03, CT06 a CT12, CT19, CT22, CT24, CT29

3 ALGORITMOS REVISADOS

3.1 Fluxograma geral do sistema

O fluxograma do menu principal elaborado na Etapa 1 foi implementado sem alterações: a função `main` exibe o menu, lê a opção, valida o intervalo de 1 a 6, executa a função correspondente e repete até que o usuário escolha encerrar.

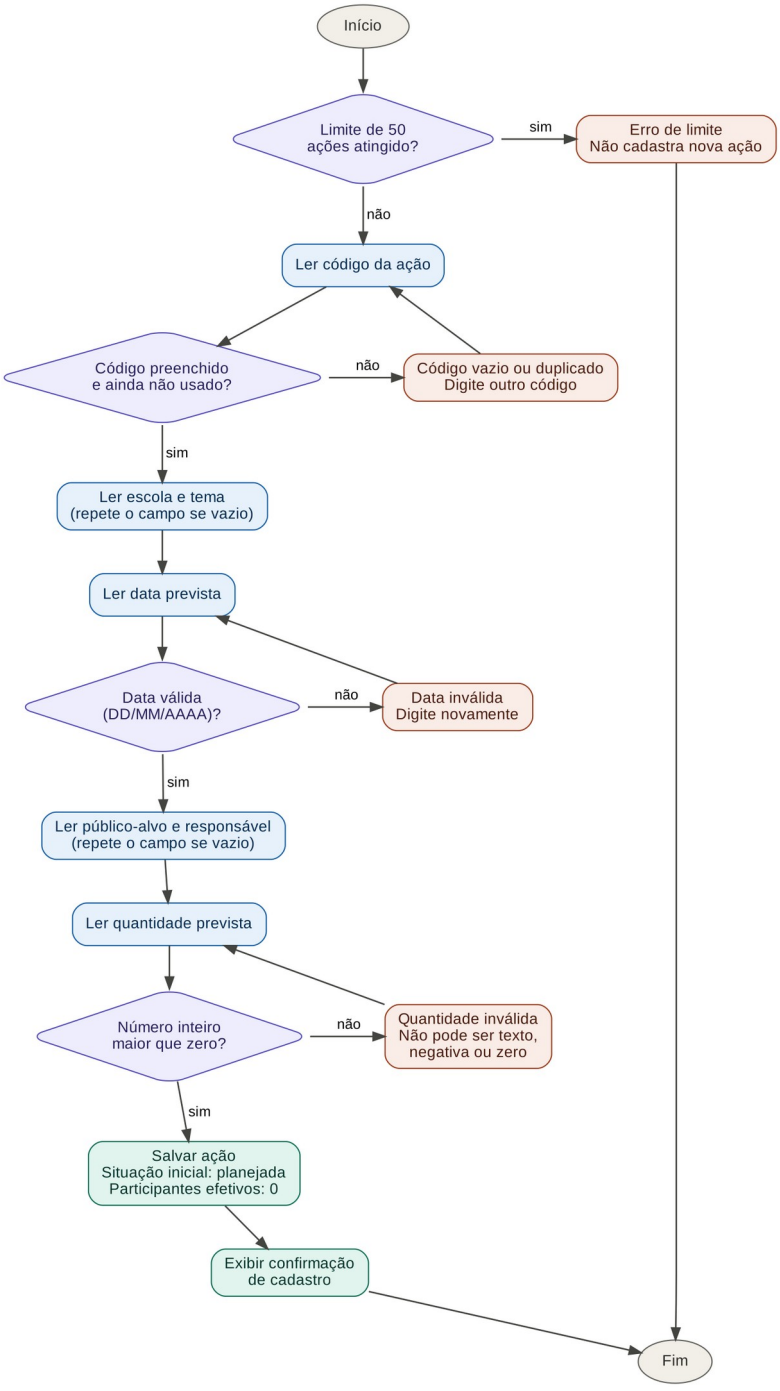
Figura 1 – Fluxograma geral do sistema (menu principal)



3.2 Fluxograma detalhado do cadastro (atualizado)

A operação detalhada escolhida pelo grupo continua sendo o cadastro de ação. O fluxograma foi atualizado para refletir a implementação: verificação do limite de armazenamento antes da leitura, repetição apenas do campo inválido e validação de data.

Figura 2 – Fluxograma do cadastro de ação de vacinação (versão implementada)



3.3 Pseudocódigos

Os pseudocódigos abaixo correspondem às funções implementadas. O pseudocódigo do menu principal permanece o mesmo da Etapa 1.

3.3.1 Cadastro de ação (atualizado)

```
INÍCIO CadastrarAcao(total)
  SE total >= 50 ENTÃO
    EXIBIR "Erro: limite de 50 ações atingido."
    RETORNAR total
  FIM SE
  REPETIR
    LER codigo   (obrigatório)
    SE codigo JÁ EXISTE NA LISTA ENTÃO EXIBIR "Erro: código já cadastrado."
  ATÉ QUE codigo NÃO SEJA VAZIO E NÃO EXISTA NA LISTA
  LER escola e tema      (repete o campo enquanto estiver vazio)
  REPETIR LER data ATÉ QUE DataValida(data)   // formato DD/MM/AAAA
  LER publicoAlvo e responsavel   (repete o campo enquanto estiver vazio)
  REPETIR LER quantidadePrevista ATÉ QUE seja inteiro E maior que zero
  acao.quantidadeEfetiva <- 0
  acao.situacao <- PLANEJADA
  ADICIONAR acao À LISTA DE AÇÕES
  EXIBIR "Ação cadastrada com sucesso."
  RETORNAR total + 1
FIM CadastrarAcao
```

3.3.2 Atualização de situação

```
INÍCIO AtualizarSituacao
  SE total == 0 ENTÃO EXIBIR "Nenhuma ação cadastrada."; RETORNAR
  LER codigo
  posicao <- BUSCAR codigo NA LISTA DE AÇÕES
  SE posicao NÃO ENCONTRADA ENTÃO
    EXIBIR "Erro: registro inexistente."; RETORNAR
  FIM SE
  EXIBIR ação encontrada
  REPETIR LER opcao ATÉ QUE opcao ESTEJA ENTRE 1 E 3
  participantes <- 0
  SE opcao == REALIZADA ENTÃO
    REPETIR LER participantes ATÉ QUE seja inteiro E maior ou igual a zero
    SE participantes > quantidadePrevista ENTÃO EXIBIR aviso
  FIM SE
  acao.situacao <- opcao
  acao.quantidadeEfetiva <- participantes
  EXIBIR "Situação atualizada."
FIM AtualizarSituacao
```

3.3.3 Resumo geral

```
INÍCIO GerarResumo
  SE total == 0 ENTÃO EXIBIR "Nenhuma ação cadastrada."; RETORNAR
  planejadas, realizadas, canceladas, participantes, previstosRealizadas <- 0
  PARA CADA acao NA LISTA DE AÇÕES FAÇA
    SE acao.situacao == PLANEJADA ENTÃO planejadas <- planejadas + 1
    SENÃO SE acao.situacao == REALIZADA ENTÃO
      realizadas <- realizadas + 1
      participantes <- participantes + acao.quantidadeEfetiva
      previstosRealizadas <- previstosRealizadas + acao.quantidadePrevista
    SENÃO canceladas <- canceladas + 1
  FIM PARA
  EXIBIR planejadas, realizadas, canceladas e participantes
  SE previstosRealizadas > 0 ENTÃO
    EXIBIR participantes * 100 / previstosRealizadas, "%"
  SENÃO EXIBIR "sem ações realizadas para calcular"
  PARA CADA escola DISTINTA NA LISTA DE AÇÕES FAÇA
    EXIBIR escola, quantidade de ações e soma dos participantes
  FIM PARA
FIM GerarResumo
```

4 ENGENHARIA DE SOFTWARE

4.1 Processo de desenvolvimento e sprints

O grupo adotou uma abordagem **incremental**, com entregas semanais organizadas em sprints, permitindo ajustes contínuos de escopo e priorização das funcionalidades essenciais antes das complementares. O planejamento das sprints é o mesmo apresentado na Etapa 1.

Etapa 1 — Documento escrito (semanas 1 a 5):

Sprint	Semana	Foco	O que foi feito
1	1	Organização inicial	Formação do grupo, levantamento das datas de entrega da AEP e definição das metas de cada sprint do cronograma.
2	2	Levantamento	Contextualização do Programa Saúde na Escola (PSE), definição do problema de pesquisa e formulação do objetivo geral do trabalho.
3	3	Levantamento	Definição de pelo menos três objetivos específicos, elaboração da justificativa acadêmica e social, padronização das referências bibliográficas e redação do planejamento das sprints semanais.
4	4	Modelagem	Elaboração do fluxograma geral do sistema, do fluxograma detalhado de uma operação, dos pseudocódigos correspondentes e definição do processo de desenvolvimento adotado pelo grupo.
5	5	Modelagem	Identificação dos usuários do sistema, definição do escopo, elaboração dos requisitos funcionais e não funcionais e criação e organização do repositório no GitHub.

Etapa 2 — Implementação e apresentação (semanas 6 a 15):

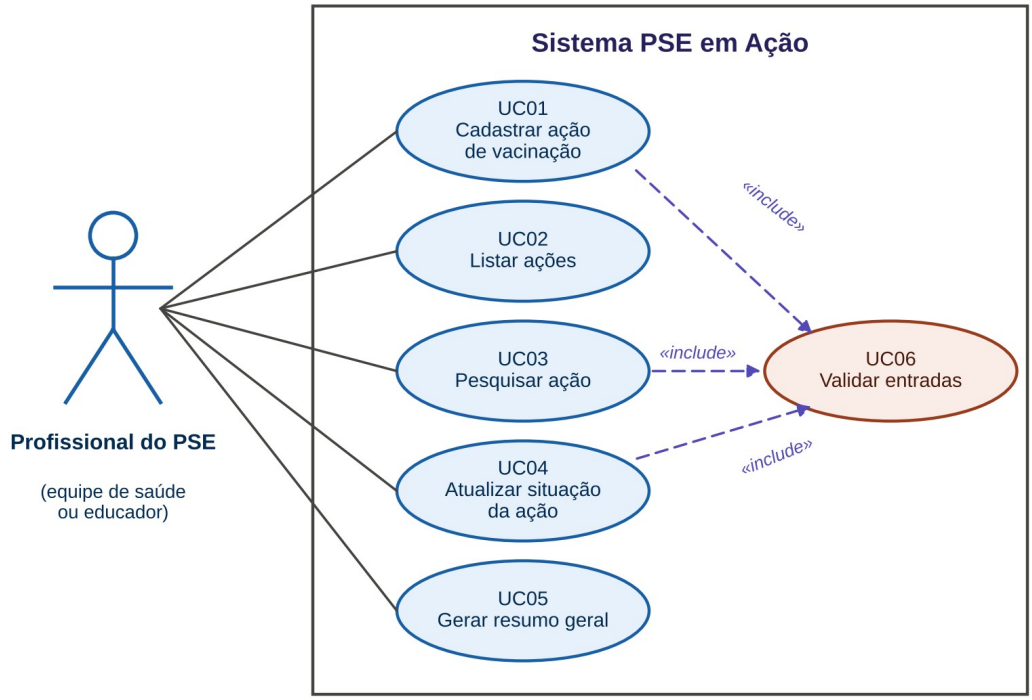
Sprint	Semana	Foco	O que foi feito
6	6	Documentação	Criação do README do repositório, identificação dos integrantes no GitHub, criação da pasta de documentação e envio da primeira entrega (Etapa 1).
7	7	Implementação	Criação da estrutura inicial do código em C, implementação do menu principal e configuração da estrutura do projeto no repositório do GitHub.
8	8	Implementação	Desenvolvimento da função de cadastro de ações de vacinação (RF01), implementação das validações de entrada e desenvolvimento da função de listagem de ações (RF02).
9	9	Implementação	Desenvolvimento da função de pesquisa por ação (RF03), da função de atualização de situação (RF04) e implementação das validações para evitar erros de código (RF06).
10	10	Implementação	Criação da função de resumo geral (RF05), revisão e aplicação de tratamento de erros, modularização completa do código em funções com responsabilidades claras e inclusão de comentários explicativos.

Sprint	Semana	Foco	O que foi feito
11	11	Testes/ Documentação	Execução de testes do sistema completo, elaboração do diagrama de casos de uso e redação das descrições dos principais casos de uso.
12	12	Testes/ Documentação	Revisão e atualização dos requisitos funcionais e não funcionais, início da escrita do manual do usuário e documentação do passo a passo para compilar e executar o programa no terminal.
13	13	Testes/ Documentação	Execução de cenários no programa com capturas de tela de cada função, inserção das capturas no manual do usuário acompanhadas de descrições explicativas e redação das considerações finais do trabalho.
14	14	Apresentação	Elaboração do roteiro do vídeo de apresentação, gravação e edição do material, com exportação para a plataforma definida.
15	15	Apresentação	Atualização do repositório do GitHub com a versão final do sistema, revisão do documento escrito e envio da entrega final da AEP.

4.2 Diagrama de casos de uso

O sistema possui um único ator, o **Profissional do PSE** (integrante da equipe de saúde ou educador), que aciona as cinco funcionalidades do menu. A validação de entradas (UC06) é incluída pelos casos de uso que recebem dados digitados (UC01, UC03 e UC04). A opção “Encerrar” do menu apenas finaliza o programa e, por isso, não foi representada como caso de uso.

Figura 3 – Diagrama de casos de uso do sistema PSE em Ação



4.3 Descrição dos casos de uso

4.3.1 UC01 — Cadastrar ação de vacinação

Objetivo	Registrar uma nova ação coletiva de vacinação no sistema.
Ator	Profissional do PSE.
Pré-condições	Sistema em execução e menu principal exibido. Quantidade de ações cadastradas menor que 50.
Fluxo principal	1. O ator escolhe a opção 1 do menu. 2. O sistema solicita o código da ação e verifica (UC06) que não está vazio nem repetido. 3. O sistema solicita escola e tema. 4. O sistema solicita a data prevista e verifica o formato DD/MM/AAAA. 5. O sistema solicita público-alvo e responsável. 6. O sistema solicita a quantidade prevista e verifica que é um inteiro maior que zero. 7. O sistema grava a ação com situação “planejada” e participantes efetivos igual a zero. 8. O sistema exibe a confirmação e retorna ao menu.
Fluxos alternativos	A1. Limite atingido (antes do passo 2): o sistema exibe o erro de limite e retorna ao menu. A2. Código vazio ou já cadastrado (passo 2): o sistema informa o erro e solicita outro código. A3. Campo obrigatório vazio (passos 3 e 5): o sistema informa o erro e solicita o mesmo campo novamente. A4. Data inválida (passo 4): o sistema informa o erro e solicita a data novamente.

	A5. Quantidade negativa, zero ou não numérica (passo 6): o sistema informa o erro e solicita novamente.
Pós-condições	A nova ação fica armazenada em memória com situação “planejada” e o total de ações aumenta em uma unidade.

4.3.2 UC02 — Listar ações

Objetivo	Apresentar de forma organizada todas as ações cadastradas.
Ator	Profissional do PSE.
Pré-condições	Sistema em execução e menu principal exibido.
Fluxo principal	1. O ator escolhe a opção 2 do menu. 2. O sistema exibe o total de ações cadastradas. 3. Para cada ação, o sistema exibe código, situação, escola, tema, data prevista, público-alvo, responsável e participantes previstos e efetivos. 4. O sistema retorna ao menu.
Fluxos alternativos	A1. Não há ações cadastradas (passo 2): o sistema exibe “Nenhuma ação cadastrada até o momento.” e retorna ao menu.
Pós-condições	Nenhum dado é alterado.

4.3.3 UC03 — Pesquisar ação

Objetivo	Localizar rapidamente ações por código, escola ou tema.
Ator	Profissional do PSE.
Pré-condições	Sistema em execução e ao menos uma ação cadastrada.
Fluxo principal	1. O ator escolhe a opção 3 do menu. 2. O sistema apresenta os critérios: 1 (código), 2 (escola), 3 (tema) ou 0 (voltar). 3. O ator escolhe o critério e informa o termo da busca. 4. O sistema percorre as ações e exibe as que correspondem (código exato; escola ou tema por trecho, sem diferenciar maiúsculas de minúsculas). 5. O sistema informa a quantidade de ações encontradas e retorna ao menu.
Fluxos alternativos	A1. Não há ações cadastradas (passo 1): o sistema informa e retorna ao menu. A2. Critério inexistente (passo 3): o sistema informa o erro e solicita o critério novamente. A3. Termo vazio (passo 3): o sistema informa que o campo é obrigatório e solicita novamente. A4. Nenhum resultado (passo 4): o sistema exibe “Nenhuma ação encontrada para ...” e retorna ao menu. A5. O ator escolhe 0 no passo 2: o sistema retorna ao menu sem pesquisar.
Pós-condições	Nenhum dado é alterado.

4.3.4 UC04 — Atualizar situação da ação

Objetivo	Alterar a situação de uma ação para planejada, realizada ou cancelada e registrar a

	quantidade efetiva de participantes quando realizada.
Ator	Profissional do PSE.
Pré-condições	Sistema em execução e ao menos uma ação cadastrada.
Fluxo principal	1. O ator escolhe a opção 4 do menu e informa o código da ação. 2. O sistema localiza a ação e exibe seus dados. 3. O ator escolhe a nova situação (1 planejada, 2 realizada, 3 cancelada). 4. Se a situação for “realizada”, o ator informa a quantidade efetiva de participantes. 5. O sistema atualiza o registro e exibe a confirmação. 6. O sistema retorna ao menu.
Fluxos alternativos	A1. Código inexistente (passo 2): o sistema exibe “registro inexistente” e retorna ao menu. A2. Opção de situação inexistente (passo 3): o sistema informa o erro e solicita novamente. A3. Quantidade negativa ou não numérica (passo 4): o sistema informa o erro e solicita novamente. A4. Quantidade efetiva maior que a prevista (passo 4): o sistema exibe um aviso e aceita o valor. A5. Situação diferente de “realizada” (passo 5): a quantidade efetiva é registrada como zero.
Pós-condições	A situação e a quantidade efetiva da ação ficam atualizadas em memória.

4.3.5 UC05 — Gerar resumo geral

Objetivo	Consolidar a quantidade de ações por situação, o total de participantes e o percentual de participação.
Ator	Profissional do PSE.
Pré-condições	Sistema em execução e ao menos uma ação cadastrada.
Fluxo principal	1. O ator escolhe a opção 5 do menu. 2. O sistema conta as ações planejadas, realizadas e canceladas. 3. O sistema soma os participantes efetivos e os participantes previstos das ações realizadas. 4. O sistema exibe o percentual de participação em relação ao previsto. 5. O sistema exibe, para cada escola, a quantidade de ações e o total de participantes. 6. O sistema retorna ao menu.
Fluxos alternativos	A1. Não há ações cadastradas (passo 2): o sistema informa que não há dados para o resumo. A2. Não há ações realizadas (passo 4): o sistema informa que não há ações realizadas para calcular o percentual.
Pós-condições	Nenhum dado é alterado.

4.3.6 UC06 — Validar entradas (incluído em UC01, UC03 e UC04)

Objetivo	Garantir que os dados digitados sejam consistentes antes de serem usados ou gravados.
Ator	Não possui ator próprio; é executado pelo sistema durante outros casos de uso.

Fluxo principal	1. O sistema recebe o texto digitado e remove espaços das pontas. 2. O sistema confere o tipo e as regras do campo: obrigatório, tamanho máximo, número inteiro, quantidade mínima, formato de data, código único e opção dentro do intervalo. 3. Se o dado for válido, o sistema o aceita e segue o caso de uso que o incluiu.
Fluxos alternativos	A1. Dado inválido (passo 2): o sistema exibe a mensagem de erro correspondente e solicita o dado novamente.
Pós-condições	Somente dados válidos chegam à memória do sistema.

5 IMPLEMENTAÇÃO EM LINGUAGEM C

5.1 Organização do repositório

O código-fonte completo está no repositório do GitHub (Seção 9), organizado da seguinte forma:

```
.
├── README.md                apresentação e instruções de uso
├── src/
│   └── main.c               código-fonte completo
├── tests/
│   └── run_tests.sh         29 testes automatizados
└── docs/
    ├── AEP_Etapa1_PSE_Vacinacao_atualizado.docx
    ├── AEP_Etapa2_PSE_Vacinacao.docx
    └── img/                 diagramas e capturas de tela
```

5.2 Estruturas de dados

As ações são guardadas em um **vetor de `struct Acao`** com 50 posições, declarado em `main` e passado às funções junto com a quantidade de ações cadastradas. A situação é representada por um `enum`.

Quadro 6 – Campos da `struct Acao`

Campo	Tipo	Descrição
`codigo`	`char[16]`	Identificador único da ação (ex.: VAC-001).
`escola`	`char[61]`	Nome da escola atendida.
`tema`	`char[61]`	Tema da ação (ex.: Vacinação contra HPV).
`data`	`char[11]`	Data prevista no formato DD/MM/AAAA.
`publicoAlvo`	`char[61]`	Descrição coletiva do público (ex.: Estudantes de 9 a 14 anos).
`responsavel`	`char[61]`	Equipe ou cargo responsável pela ação.
`quantidadePrevista`	`int`	Participantes previstos (maior que zero).
`quantidadeEfetiva`	`int`	Participantes efetivos (zero até a ação ser realizada).
`situacao`	`enum Situacao`	`PLANEJADA` (1), `REALIZADA` (2) ou `CANCELADA` (3).

5.3 Funções e responsabilidades

Quadro 7 – Funções do programa

Função	Responsabilidade	Requisito
`main`	Controla o laço do menu, lê a opção, valida o intervalo e chama a função correspondente.	Menu (Etapa 1)

Função	Responsabilidade	Requisito
`exibirMenu`	Mostra as seis opções do menu principal.	Menu (Etapa 1)
`cadastrarAcao`	Verifica o limite, coleta os dados validados e grava a nova ação.	RF01
`listarAcoes`, `exibirAcao`	Percorrem o vetor e mostram cada ação de forma organizada.	RF02
`pesquisarAcao`	Pede o critério e o termo e exibe as ações encontradas.	RF03
`atualizarSituacao`	Localiza a ação, altera a situação e registra os participantes efetivos.	RF04
`gerarResumo`	Conta ações por situação, soma participantes, calcula o percentual e o total por escola.	RF05
`lerLinha`	Lê uma linha do teclado, descarta o excesso e finaliza o programa se a entrada acabar.	RF06
`lerTextoObrigatorio`	Lê um texto não vazio e dentro do tamanho permitido, repetindo até ser válido.	RF06
`lerInteiro`, `lerQuantidade`	Convertem e validam números inteiros (mínimo 0 ou 1 e máximo de 1.000.000).	RF06
`lerCodigoNovo`, `lerDataPrevista`	Repetem a leitura até o código ser inédito e a data ser válida.	RF01, RF06
`dataValida`	Confere o formato, o mês, os dias de cada mês e o ano bissexto.	RF06
`buscarPorCodigo`	Retorna a posição de uma ação pelo código, ou -1 se não existir.	RF03, RF04, RF06
`igualIgnorandoCaixa`, `contemTexto`	Comparam textos e buscam trechos sem diferenciar maiúsculas de minúsculas.	RF03
`removerEspacosDasPontas`	Limpa espaços e quebras de linha no início e no fim do texto.	RF06
`nomeSituacao`	Converte o `enum` em texto para exibição.	RF02

Todas as funções recebem os dados de que precisam por parâmetro (vetor e quantidade de ações), sem variáveis globais. `cadastrarAcao` devolve a nova quantidade de ações, o que dispensa o uso de ponteiros nesse ponto.

5.4 Validações e tratamento de erros

Quadro 8 – Situações tratadas e mensagens exibidas

Situação	Tratamento e mensagem
Opção de menu inexistente, vazia ou não numérica	Volta ao menu: “Erro: opção inválida. Digite um número de 1 a 6.”
Campo obrigatório vazio	Repete o campo: “Erro: este campo é obrigatório e não pode ficar vazio.”
Texto maior que o permitido	Repete o campo: “Erro: texto muito longo (máximo de N caracteres).”
Código duplicado	Repete o código: “Erro: já existe uma ação cadastrada com este código.”

Situação	Tratamento e mensagem
Data inválida	Repete a data: “Erro: data inválida. Use o formato DD/MM/AAAA (ex.: 15/10/2026).”
Quantidade não numérica	Repete a quantidade: “Erro: digite um número inteiro válido.”
Quantidade negativa	Repete a quantidade: “Erro: a quantidade não pode ser negativa.”
Quantidade prevista igual a zero	Repete a quantidade: “Erro: a quantidade prevista deve ser maior que zero.”
Registro inexistente	Volta ao menu: “Erro: registro inexistente. Nenhuma ação com o código ...”
Critério de pesquisa ou situação inexistente	Repete a escolha: “Erro: opção inválida. Digite um número de 0 a 3.” ou “Digite 1, 2 ou 3.”
Limite de armazenamento atingido	Volta ao menu: “Erro: limite de 50 ações atingido. Não é possível cadastrar novas ações.”
Participantes efetivos acima do previsto	Aceita e avisa: “Aviso: a quantidade efetiva é maior que a prevista (N).”
Lista vazia (listar, pesquisar, atualizar ou resumir)	Mensagem informativa e retorno ao menu.
Fim da entrada de dados (Ctrl+D ou Ctrl+Z)	Encerra com a mensagem “Entrada encerrada. Finalizando o sistema.”, evitando laço infinito.

6 TESTES

6.1 Ambiente e método

Os testes foram de **caixa-preta**: para cada cenário, entradas foram enviadas ao programa em execução e a saída foi comparada com a mensagem esperada. A bateria está automatizada no arquivo `tests/run_tests.sh` do repositório, que compila o programa e executa os 29 casos. Ambiente: Ubuntu 24.04, gcc 13.3.0, padrão C99.

Além dos testes funcionais, o código foi compilado com `-Wall -Wextra -pedantic` sem nenhum aviso e a bateria foi repetida com o programa compilado com AddressSanitizer e UndefinedBehaviorSanitizer (`-fsanitize=address,undefined`), sem erros de memória ou comportamento indefinido.

6.2 Casos de teste e resultados

Quadro 9 – Registro dos testes executados

ID	Cenário	Entrada	Resultado esperado	Resultado
CT01	Opção de menu fora do intervalo	Digitar 9	Erro “opção inválida” e menu exibido novamente	**Aprovado**
CT02	Opção de menu não numérica	Digitar abc	Erro “opção inválida”	**Aprovado**
CT03	Opção de menu vazia	Apenas Enter	Erro “opção inválida”	**Aprovado**
CT04	Encerrar o sistema	Opção 6	Mensagem “Encerrando o sistema...” e fim do programa	**Aprovado**
CT05	Cadastro válido	Todos os campos corretos	Confirmação com situação inicial “planejada”	**Aprovado**
CT06	Código duplicado	VAC-001 e depois vac-001	Erro de código já cadastrado; pede novo código	**Aprovado**
CT07	Campo obrigatório vazio	Enter no campo código	Erro de campo obrigatório	**Aprovado**
CT08	Quantidade prevista negativa	-5	Erro “não pode ser negativa”	**Aprovado**
CT09	Quantidade prevista zero	0	Erro “deve ser maior que zero”	**Aprovado**
CT10	Quantidade prevista não numérica	abc	Erro “digite um número inteiro válido”	**Aprovado**
CT11	Data inexistente	31/02/2026	Erro de data inválida	**Aprovado**
CT12	Data em formato errado	2026-10-15	Erro de data inválida	**Aprovado**
CT13	Listar sem ações	Opção 2 sem cadastros	Mensagem “Nenhuma ação cadastrada”	**Aprovado**
CT14	Listar com ações	Opção 2 com 2 ações	Total (2) e dados das duas ações	**Aprovado**
CT15	Pesquisar por código	Critério 1, “vac-002”	Encontra VAC-002 (1 ação)	**Aprovado**

ID	Cenário	Entrada	Resultado esperado	Resultado
CT16	Pesquisar por escola (trecho)	Critério 2, “monteiro”	Encontra a ação da E.M. Monteiro Lobato	**Aprovado**
CT17	Pesquisar por tema	Critério 3, “caderneta”	Encontra VAC-002	**Aprovado**
CT18	Pesquisar sem resultado	Critério 1, “VAC-999”	Mensagem “Nenhuma ação encontrada”	**Aprovado**
CT19	Critério de pesquisa inválido	Digitar 7	Erro de opção; pede o critério novamente	**Aprovado**
CT20	Atualizar código inexistente	VAC-999	Erro “registro inexistente”	**Aprovado**
CT21	Atualizar para realizada	VAC-001, 2, 98	Situação REALIZADA registrada	**Aprovado**
CT22	Participantes efetivos negativos	-3 e depois 98	Erro e, em seguida, atualização concluída	**Aprovado**
CT23	Atualizar para cancelada	VAC-001, 3	Situação CANCELADA registrada	**Aprovado**
CT24	Situação inexistente	Digitar 7	Erro “Digite 1, 2 ou 3”	**Aprovado**
CT25	Efetivos acima do previsto	130 (previsto 120)	Aviso e registro da quantidade	**Aprovado**
CT26	Resumo sem ações	Opção 5 sem cadastros	Mensagem de que não há dados	**Aprovado**
CT27	Resumo com dados	1 realizada (90 de 120) e 1 cancelada	Contagens corretas, 90 participantes e 75,0%	**Aprovado**
CT28	Resumo sem ações realizadas	Apenas ação planejada	Mensagem “sem ações realizadas para calcular”	**Aprovado**
CT29	Limite de armazenamento	51 cadastros seguidos	Erro de limite de 50 ações no 51º cadastro	**Aprovado**

Resultado geral: 29 casos executados, 29 aprovados e nenhum reprovado. Os testes cobrem as seis funcionalidades mínimas e os tratamentos de registro inexistente, código duplicado e limite de armazenamento exigidos para a Etapa 2.

7 MANUAL DO USUÁRIO

7.1 Objetivo

O **PSE em Ação** ajuda a equipe escolar e de saúde a organizar as ações coletivas de vacinação do Programa Saúde na Escola: cadastrar o que foi planejado, consultar ações por código, escola ou tema, registrar se a ação foi realizada ou cancelada e acompanhar a participação. O programa usa somente dados fictícios e coletivos. **Não digite nomes, diagnósticos ou qualquer dado individual de estudantes.**

7.2 Requisitos para uso

a) computador com Windows, Linux ou macOS;
b) compilador C (gcc) com suporte ao padrão C99, ou uma IDE que o utilize (por exemplo, Code::Blocks);
c) terminal (Prompt de Comando, PowerShell, Terminal do Linux ou do macOS).

7.3 Compilação e execução

Abra o terminal na pasta do projeto e execute os comandos conforme o sistema operacional.

7.3.1 Linux e macOS

```
gcc -Wall -Wextra -std=c99 -o pse_em_acao src/main.c
./pse_em_acao
```

7.3.2 Windows (MinGW ou MSYS2)

```
gcc -Wall -Wextra -std=c99 -o pse_em_acao.exe src/main.c
pse_em_acao.exe
```

Em uma IDE, abra o arquivo `src/main.c`, compile e execute pelo próprio ambiente. Se os acentos aparecerem incorretos no Windows, execute `chcp 65001` no terminal antes de iniciar o programa. Para executar os testes automatizados (Linux, macOS ou Git Bash), use `bash tests/run_tests.sh`.

7.4 Opções do menu e exemplos de uso

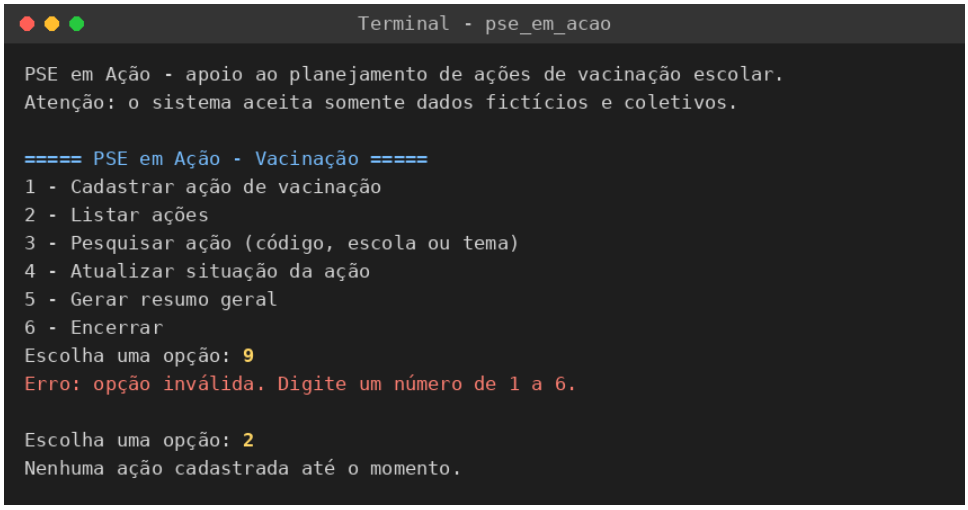
Quadro 10 – Opções do menu principal

Opção	Função	Como usar
1	Cadastrar ação de vacinação	Informe código, escola, tema, data (DD/MM/AAAA), público-alvo, responsável e quantidade prevista.
2	Listar ações	Exibe todas as ações cadastradas. Não pede nenhum dado.
3	Pesquisar ação	Escolha o critério (1 código, 2 escola, 3 tema, 0 voltar) e digite o termo. Escola e tema aceitam apenas um trecho.
4	Atualizar situação	Informe o código, escolha 1 (planejada), 2 (realizada) ou 3 (cancelada) e, se realizada, a quantidade efetiva de participantes.
5	Gerar resumo geral	Exibe ações por situação, total de participantes, percentual de participação e total por escola.
6	Encerrar	Finaliza o programa. Os dados em memória são perdidos.

7.4.1 Menu principal e opção inválida

Ao iniciar, o programa exibe um aviso sobre o uso de dados fictícios e coletivos e o menu. Uma opção fora do intervalo gera mensagem de erro e o menu é exibido novamente. A listagem sem ações cadastradas informa que ainda não há registros (Figura 4).

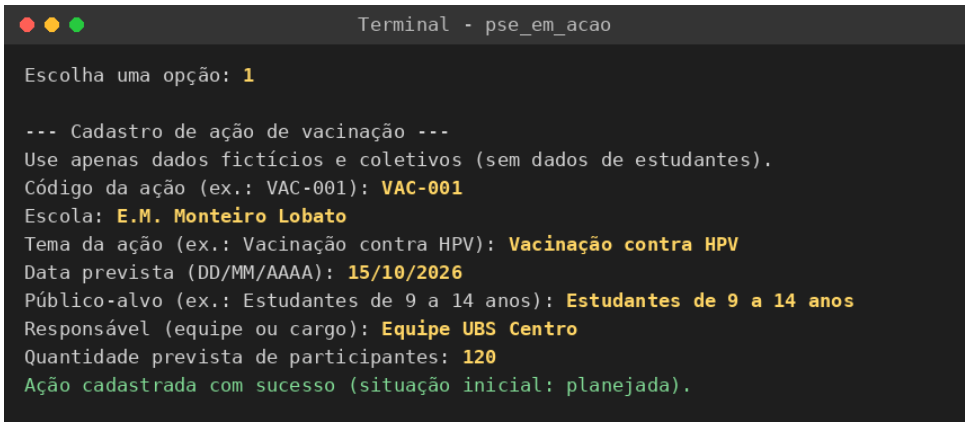
Figura 4 – Menu principal, opção inválida e listagem sem ações



7.4.2 Cadastro de ação (opção 1)

O sistema pergunta um campo por vez e confirma o cadastro ao final. A ação nasce com a situação “planejada”.

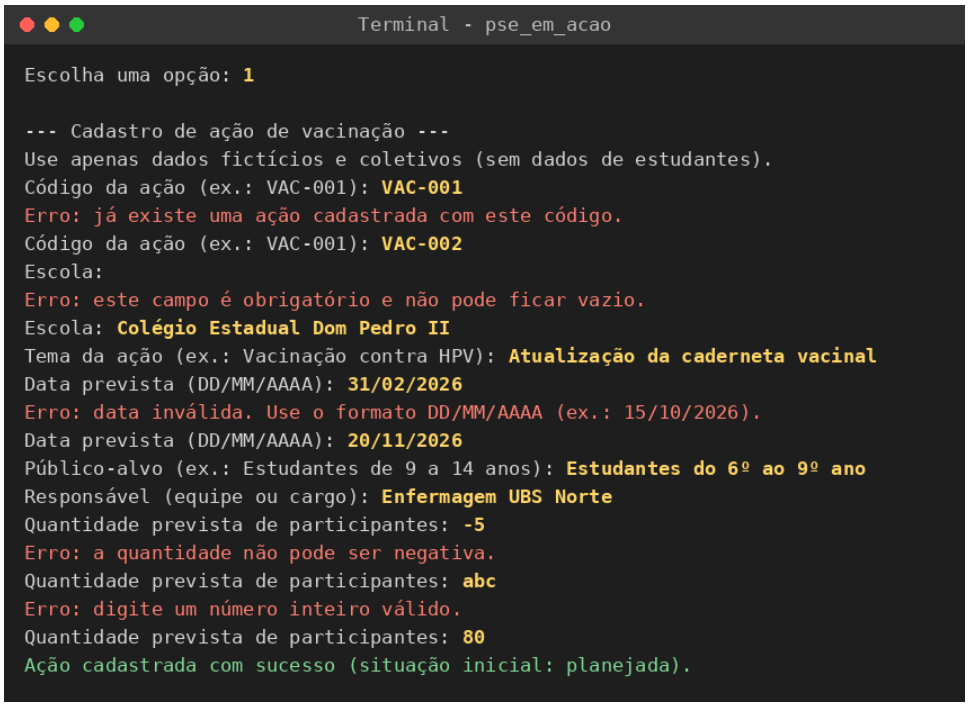
Figura 5 – Cadastro de uma ação de vacinação



7.4.3 Validações do cadastro

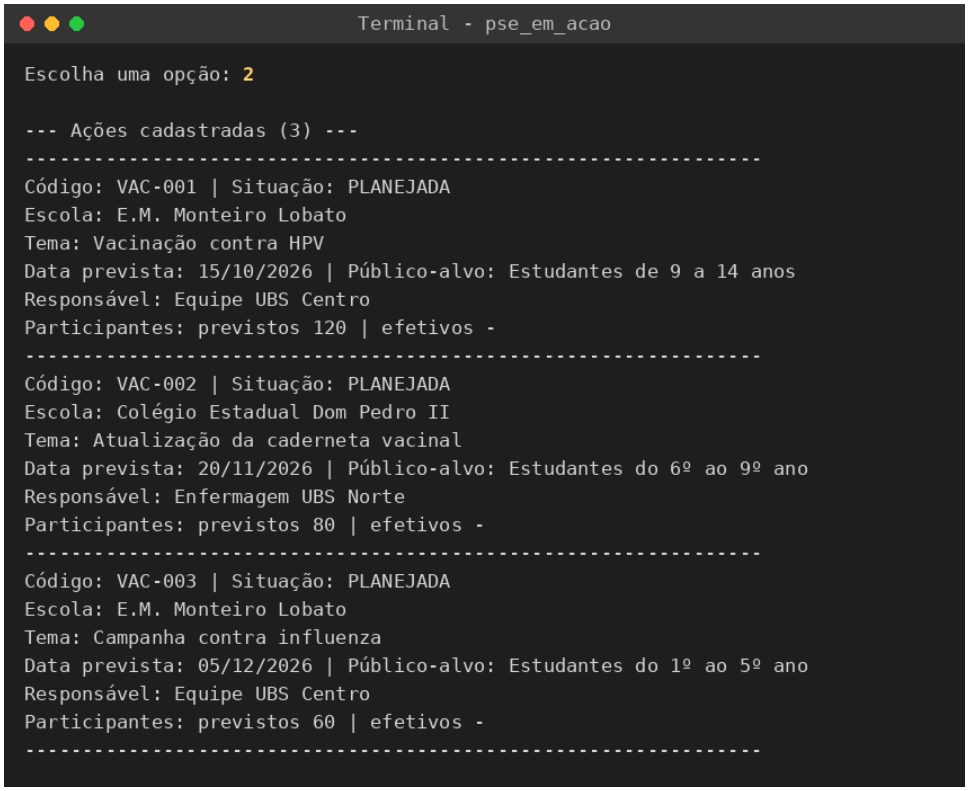
Quando um dado é inválido, o programa explica o problema e pede somente aquele campo novamente. A Figura 6 mostra código repetido, campo vazio, data inexistente, quantidade negativa e quantidade não numérica.

Figura 6 – Mensagens de erro durante o cadastro



7.4.4 Listagem (opção 2)

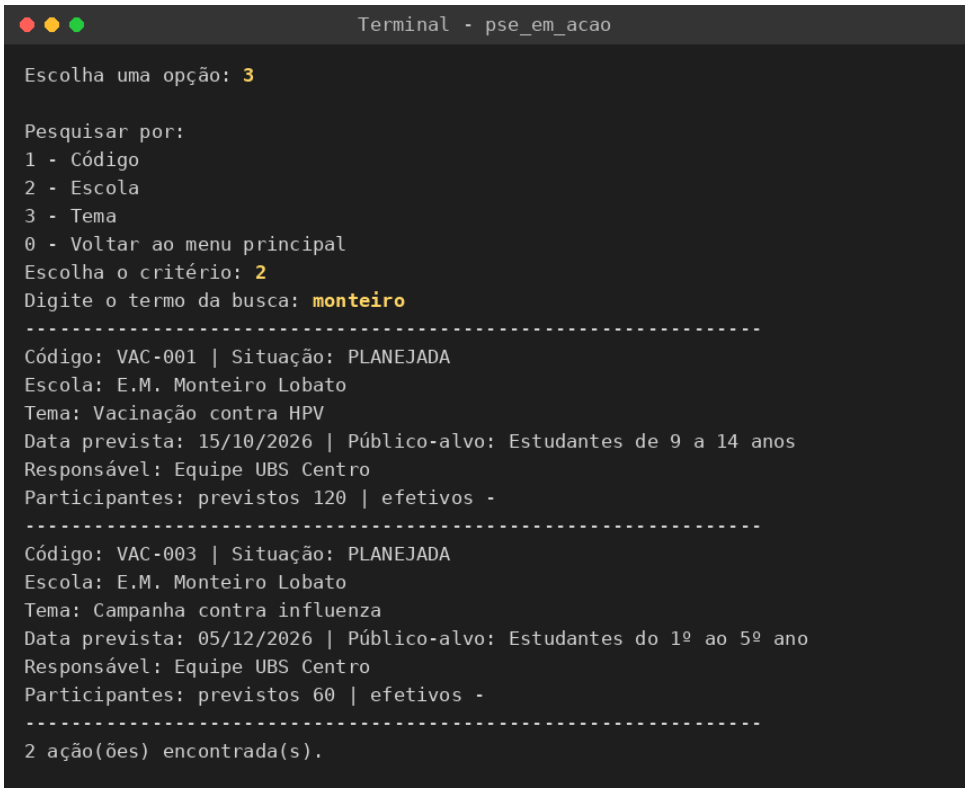
Figura 7 – Listagem das ações cadastradas



7.4.5 Pesquisa (opção 3)

A pesquisa por escola aceita um trecho do nome e não diferencia maiúsculas de minúsculas. No exemplo, o termo “monteiro” localiza as duas ações da E.M. Monteiro Lobato.

Figura 8 – Pesquisa de ações por escola



Quando nada é encontrado, ou quando o código informado para atualização não existe, o programa avisa e volta ao menu (Figura 9).

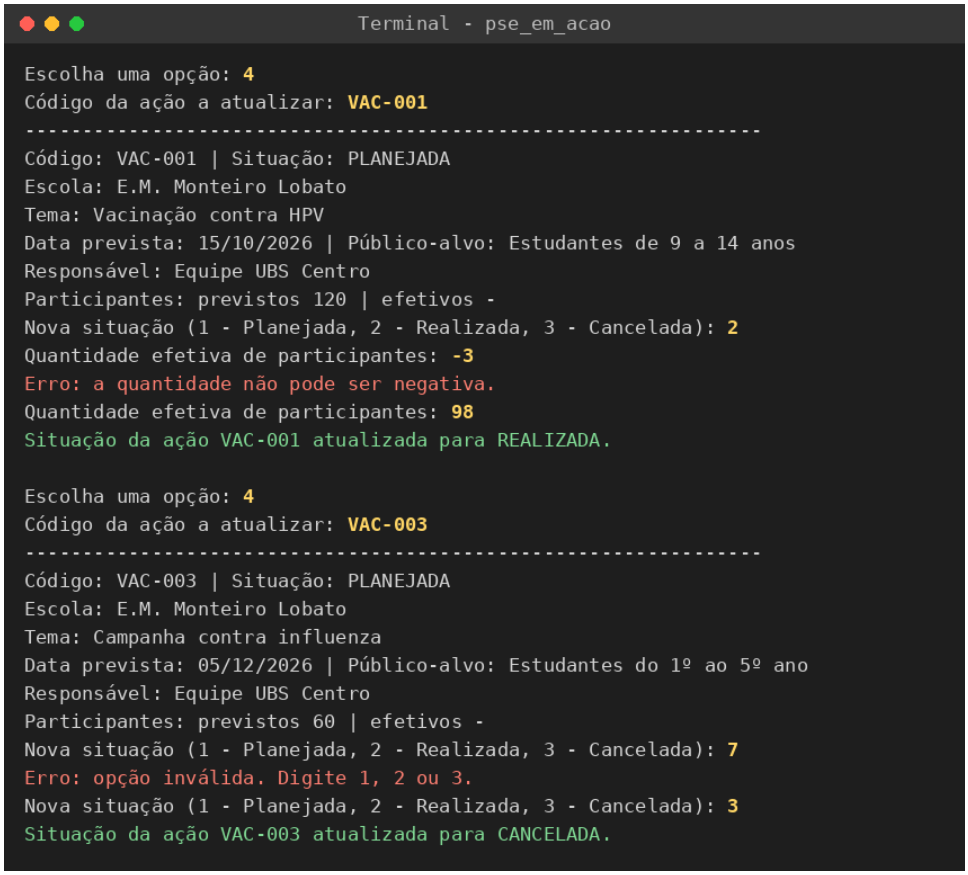
Figura 9 – Pesquisa sem resultado e atualização de código inexistente



7.4.6 Atualização de situação (opção 4)

Depois de informar o código, o programa mostra a ação encontrada e pede a nova situação. Para “realizada”, é obrigatório informar a quantidade efetiva de participantes, que não pode ser negativa. Ao cancelar, a quantidade efetiva fica zerada.

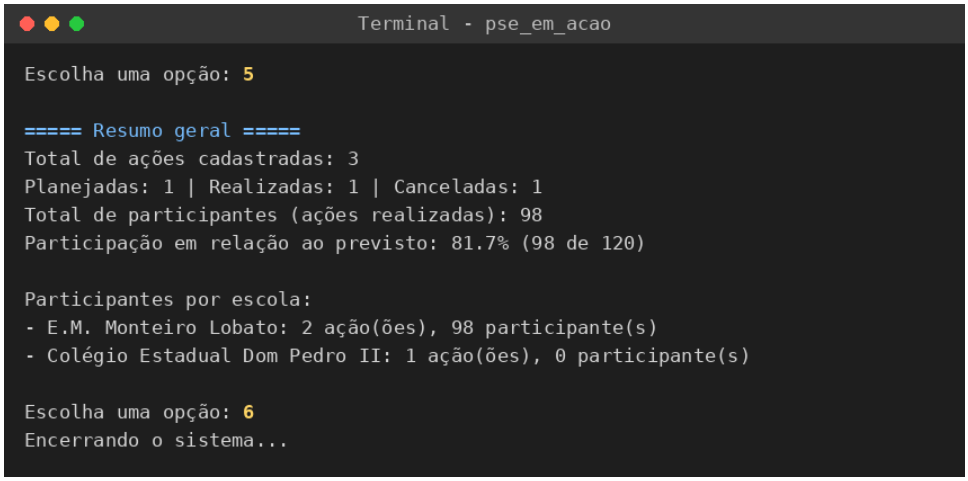
Figura 10 – Atualização de situação para realizada e cancelada



7.4.7 Resumo geral (opção 5) e encerramento (opção 6)

O resumo mostra as ações por situação, o total de participantes das ações realizadas e o percentual de participação em relação ao previsto dessas mesmas ações (98 de 120, ou 81,7%), além do total por escola.

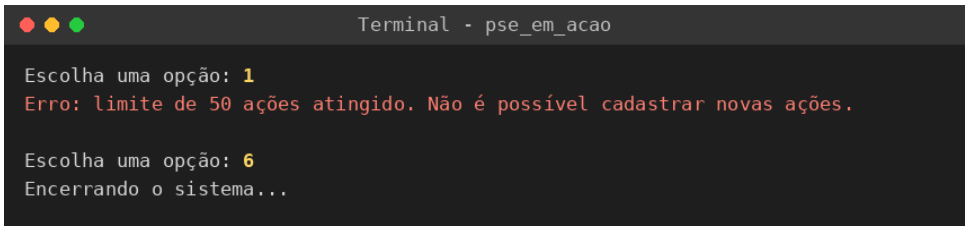
Figura 11 – Resumo geral e encerramento do programa



7.4.8 Limite de armazenamento

O programa guarda até 50 ações por execução. Ao tentar cadastrar a 51ª, é exibida uma mensagem de erro e nenhum dado é perdido.

Figura 12 – Tentativa de cadastro com o limite de 50 ações atingido (trecho final da sessão)



7.5 Dúvidas frequentes

Quadro 11 – Problemas comuns e como resolver

Problema	O que fazer
Os acentos aparecem errados no Windows.	Execute `chcp 65001` no terminal e abra o programa novamente.
“gcc” não é reconhecido como comando.	Instale o gcc (MinGW ou MSYS2 no Windows) ou compile pela IDE.
As ações sumiram depois de fechar o programa.	Os dados ficam apenas em memória. Registre o que for necessário em um documento oficial antes de encerrar.

Problema	O que fazer
A pesquisa por escola não encontrou uma escola que existe.	Digite um trecho do nome sem acento ou confira a grafia usada no cadastro.
A escola aparece duas vezes no resumo.	O nome foi cadastrado com grafias diferentes. Use sempre o mesmo nome para cada escola.
Não consigo cadastrar mais ações.	O limite de 50 ações foi atingido. Encerre e reinicie o programa para começar um novo registro.

8 CONSIDERAÇÕES FINAIS

8.1 Resultados alcançados

O grupo desenvolveu uma aplicação em C que cumpre o escopo mínimo do desafio: cadastra, lista, pesquisa e atualiza ações de vacinação do PSE, registra participantes e gera um resumo com o percentual de participação, com validação das entradas. A solução respeita o nível do segundo semestre, pois usa `struct`, vetores, funções, condicionais e laços, sem banco de dados, interface gráfica ou bibliotecas avançadas. Os 29 testes executados foram aprovados e o código compila sem avisos.

Houve coerência entre o planejamento e a implementação: o menu e o fluxograma geral da Etapa 1 foram implementados sem mudanças, e as alterações no cadastro e nos requisitos foram registradas e justificadas na Seção 2.4.

8.2 Reflexão social e ética

Quem utiliza. O sistema é voltado a profissionais da Atenção Primária e a educadores que organizam as ações do PSE em conjunto, o que responde à fragilidade apontada por Sousa, Esperidião e Medina (2017) quanto à pouca sistematização das ações. Ter em um só lugar o que foi planejado, realizado ou cancelado facilita a conversa entre os setores.

Dificuldades de uso. A interface em terminal exige familiaridade mínima com o computador, e os dados em memória se perdem ao fechar o programa. A qualidade do resumo por escola também depende de a equipe escrever sempre o mesmo nome para a mesma escola.

Apoio ao planejamento. As informações permitem ver quais ações ainda estão planejadas, quais foram canceladas e qual foi a adesão em relação ao previsto, ajudando a decidir onde reforçar a divulgação ou reagendar atividades.

Limites éticos. Os dados de saúde são dados pessoais sensíveis segundo a Lei Geral de Proteção de Dados (BRASIL, 2018). Por isso, o sistema trabalha apenas com informações coletivas e fictícias, não pede nome, diagnóstico ou situação vacinal individual, não faz triagem nem recomendação, e alerta o operador sobre isso no início do programa e no cadastro. A ferramenta organiza informações; as decisões de saúde continuam com os profissionais.

8.3 Limitações

a) os dados não são gravados em arquivo e se perdem ao encerrar o programa;
b) a capacidade é de 50 ações por execução;
c) depois de cadastrada, a ação só pode ter a situação alterada; não há edição de outros campos nem exclusão;

d) a busca e a comparação sem diferenciar maiúsculas de minúsculas funcionam apenas para letras sem acento;
e) a data não é comparada com a data atual, portanto aceita datas passadas;
f) não há controle de acesso: qualquer pessoa que abra o programa pode usá-lo.

8.4 Aprendizados

O projeto permitiu praticar, em um problema real, os conteúdos do semestre: modelar dados com `struct` e `enum`, armazenar registros em vetor, dividir a solução em funções pequenas, controlar o fluxo com condicionais e laços e, principalmente, tratar entradas inválidas. Ficou claro que a validação de dados dá mais trabalho do que as funcionalidades em si e que um bom conjunto de testes permite alterar o código com segurança. A abordagem incremental e a rastreabilidade entre requisitos, fluxogramas, código e testes ajudaram a manter o projeto coerente.

8.5 Possibilidades de melhoria

a) gravar e ler as ações em arquivo, para que os dados não se percam ao encerrar;
b) permitir editar e excluir ações cadastradas;
c) usar alocação dinâmica de memória para remover o limite fixo de 50 ações;
d) filtrar e ordenar a listagem por data, escola ou situação;
e) exportar o resumo em arquivo de texto ou CSV para compartilhar com a equipe intersetorial;
f) cadastrar a lista de escolas para evitar variações de grafia.

9 REPOSITÓRIO E VÍDEO DE APRESENTAÇÃO

Repositório no GitHub: https://github.com/MatheusDona/AEP_Segundo_Semestre.git
Vídeo de apresentação (máximo de 7 minutos, acesso liberado para avaliação): [inserir link do vídeo]

REFERÊNCIAS

BIBLIOTECA VIRTUAL EM SAÚDE (BVS/MS). **Verificação da Situação Vacinal — Caderno Temático PSE**. Brasília, DF: Ministério da Saúde, [s.d.]. Disponível em: https://bvsms.saude.gov.br/bvs/publicacoes/caderno_tematico_pse_situacao_vacinal.pdf. Acesso em: 10 set. 2026.

BRASIL. **Decreto nº 6.286, de 5 de dezembro de 2007**. Institui o Programa Saúde na Escola — PSE e dá outras providências. Diário Oficial da União: Brasília, DF, 6 dez. 2007.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: Brasília, DF, 15 ago. 2018.

SOUSA, Marta Caires de; ESPERIDIÃO, Monique Azevedo; MEDINA, Maria Guadalupe. A intersetorialidade no Programa Saúde na Escola: avaliação do processo político-gerencial e das práticas de trabalho. **Ciência & Saúde Coletiva**, Rio de Janeiro, v. 22, n. 6, p. 1781-1790, 2017. DOI: 10.1590/1413-81232017226.24262016.